

VIT BHOPAL UNIVERSITY

School of Computing Science & Engineering

PROJECT REPORT

“Spam Email Detector System”

(A Streamlit-based Spam Email Detection System)

Submitted By: Siya Panwar

Reg. No.: 25BAI10149

Branch: B.Tech – Computer Science & Engineering(AIML)

Course Code: CSA2001

Course Name: Fundamentals of Artificial Intelligence and Machine learning

Submitted To: Dr Suneet Joshi

School of Computing Science & Engineering

Introduction

Email has become one of the most widely used communication tools in both personal and professional environments. However, with its growing usage comes the increasing threat of spam emails, which often contain unwanted advertisements, phishing attempts, or malicious content. Relying solely on manual identification of spam is inefficient and error-prone, as users may overlook subtle indicators or fall victim to deceptive messages. This creates a strong need for an automated system that can accurately distinguish between legitimate and spam emails.

A Spam Email Detection System is an intelligent solution designed to address this issue. It utilizes technologies such as Python and machine learning algorithms, along with libraries like Scikit-learn, Pandas, and Natural Language Processing (NLP) tools, to analyze email content and classify messages as spam or not spam. By examining features such as keywords, email structure, frequency patterns, and sender information, the system learns to identify patterns commonly associated with spam emails.

This system allows users to automatically filter and manage incoming emails through an efficient and user-friendly interface. By providing accurate and real-time detection, it minimizes the risk of security threats, reduces inbox clutter, and enhances overall productivity. Ultimately, the project improves digital communication by ensuring a safer and more organized email experience.

Problem Statement

With the rapid increase in digital communication, users are frequently exposed to a large number of spam emails, including unsolicited advertisements, phishing attempts, and malicious content. Identifying such emails manually is time-consuming and often unreliable, as spam messages are becoming increasingly sophisticated and harder to distinguish from legitimate ones. This leads to cluttered inboxes, reduced productivity, and potential security risks such as data breaches and financial loss.

Therefore, there is a need for an automated and intelligent system that can accurately detect and filter spam emails. Such a system should efficiently analyze email content and patterns to minimize errors and ensure a safer, more organized email experience for users.

Project Objectives

- **User Interface Objective:** To design a simple, clean, and user-friendly interface using Python (and frameworks like Streamlit or Tkinter) that allows users to input or upload emails and easily check whether they are spam or not.
- **Data Collection Objective:** To utilize reliable datasets (such as labeled spam/ham email datasets) for training and testing the system, ensuring accurate and effective model performance.
- **System Functionality Objective:** To develop a machine learning-based system that analyzes email content and classifies messages as spam or non-spam in real time with high accuracy.
- **Text Processing Objective:** To implement Natural Language Processing (NLP) techniques such as tokenization, stop-word removal, and vectorization to efficiently process and extract meaningful features from email text.
- **Model Optimization Objective:** To apply and compare different machine learning algorithms (e.g., Naïve Bayes, Logistic Regression) and optimize the model to improve classification accuracy and reduce false positives/negatives.
- **User Experience Objective:** To ensure a responsive and interactive system that provides clear results and feedback, helping users quickly understand the classification outcome.
- **Error Handling Objective:** To implement robust error handling for invalid inputs, unsupported file formats, or missing data, ensuring smooth system performance.
- **Data Management Objective:** To enable storage or export of classification results (e.g., CSV/JSON), allowing users to review and analyze detected spam emails if required.

Functional Requirements

Functional requirements define the specific actions or tasks the system must perform.

1. Data Handling & Input

- **User Input Handling** (manual email text input or file upload)
- **Dataset Handling** (loading pre-labeled spam/ham datasets for training/testing)
- **Text Preprocessing** (cleaning text, removing stop words, tokenization)

2. Spam Detection & Processing

- **Email Classification** (identify whether an email is spam or not spam)
- **Feature Extraction** (convert text into numerical form using techniques like TF-IDF or Bag of Words)
- **Model Prediction** (apply trained machine learning model for classification)
- **Pattern Analysis** (detect spam keywords, suspicious phrases, or abnormal structures)

3. Output & Result Display

- **Classification Result Display** (show whether the email is Spam or Not Spam)
- **Confidence Score Display** (probability or accuracy of prediction)
- **Highlighted Keywords** (optional highlighting of suspicious/spam words)
- **Structured Output Format** (clear sections or cards for results)
- **Readable and Clear Visualization**

4. User Interface (UI) Requirements

- **Email Input Field / Upload Option**
- **Analyze / Detect Spam Button**
- **Dynamic Result Display Panel**
- **Error Messages Display** (empty input, invalid file format, processing error)
- **Responsive and Interactive Design**

5. System Reliability & Validation

- **Input Validation** (check for empty or invalid email content)
- **Model Error Handling** (handle prediction errors or model loading issues)
- **File Validation** (supported formats like .txt, .csv)
- **Loading Indicator** (while processing email data)

Non-Functional Requirements

Non-functional requirements describe constraints and quality attributes of the system.

1. Performance

- **Response Time:** The system should classify an email as spam or not spam within 1–2 seconds after the user submits the input, depending on system resources and model complexity.

2. Usability (UX)

- **Error Handling:** The system must display clear and user-friendly error messages for empty inputs, unsupported file formats, or processing failures, ensuring ease of use for all users.

3. Reliability

- **Availability:** The application should remain stable and consistently available during usage, handling model errors or unexpected inputs gracefully without crashing.

4. Maintainability

- **Code Documentation:** All major components (such as data preprocessing, model training, and prediction modules) should include proper comments or docstrings to ensure easy understanding and future updates.

5. Portability

- **Platform Compatibility:** The system should run smoothly across different environments (Windows, macOS, Linux) and function correctly in modern web browsers if deployed as a web application.

6. Scalability

- **Handling Multiple Inputs:** The system should efficiently handle multiple email analyses or batch processing without significant performance degradation, ensuring consistent results even with increased usage.

Design Decisions & Rationale

Why Python?

- Fast for prototyping
- Streamlit simplifies UI development

Why Streamlit?

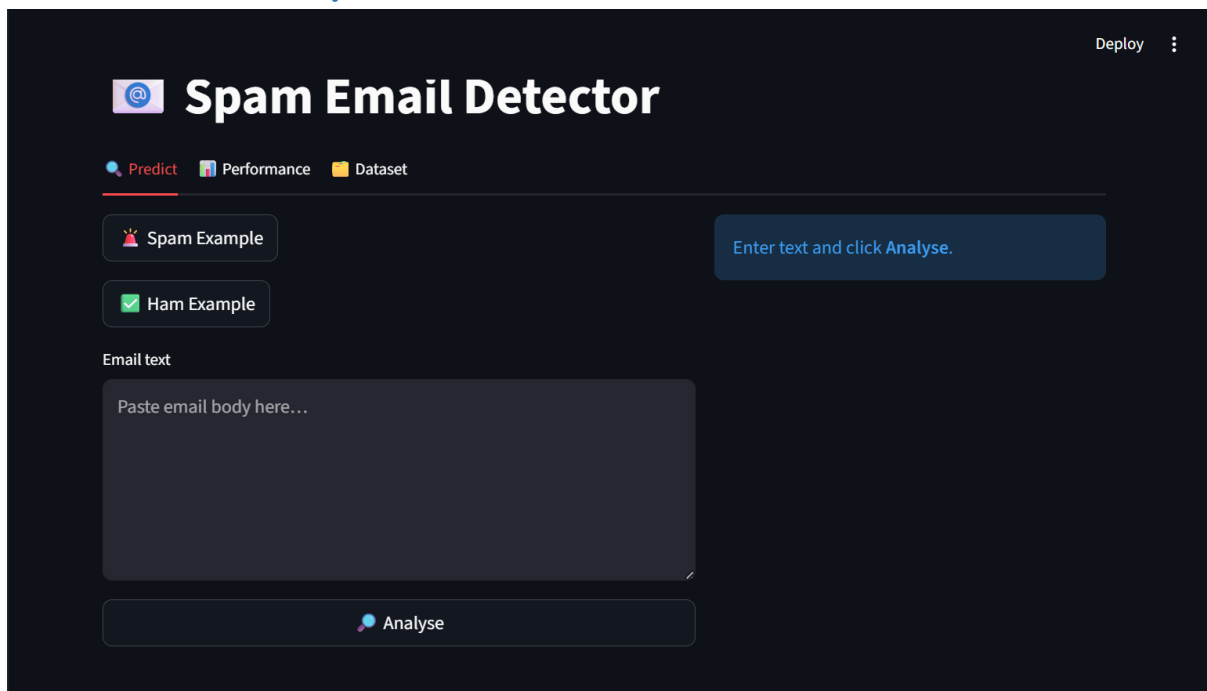
- No frontend coding required
- Real-time auto-updating interface

Implementation Details

File	Purpose
1) Streamlit (<code>streamlit</code>)	• Used to build the web-based user interface of the weather application. • Helps create input fields, buttons, layouts, and display results interactively.
2) Pandas (<code>pandas</code>)	• Used for data handling and manipulation • Creates and manages datasets (DataFrames) • Reads CSV files and prepares data for training and prediction
3) NumPy (<code>numpy</code>)	• Used for numerical operations • Helps in calculations like probability/confidence scores • Supports efficient array and mathematical computations
4) Matplotlib (<code>matplotlib.pyplot</code>)	• Used to create graphs and charts • Displays model performance visually

5) Seaborn (<code>seaborn</code>)	• Used for advanced visualization • Creates heatmaps (e.g., confusion matrix) with better styling
6) Train-Test Split	<pre>from sklearn.model_selection import train_test_split</pre> • Splits dataset into training and testing sets
7) String Module (<code>string</code>)	• Used to remove punctuation from text during preprocessing • Helps clean email content before analysis

Screenshots / Results



The screenshot shows a web application titled "Spam Email Detector". At the top right, there is a "Deploy" button and a menu icon. Below the title, there are three tabs: "Predict" (active), "Performance", and "Dataset". Under the "Predict" tab, there are two buttons: "Spam Example" (with a warning icon) and "Ham Example" (with a checkmark icon). To the right of these buttons is a large blue button that says "Enter text and click Analyse.". Below these buttons is a text input area labeled "Email text" with a placeholder "Paste email body here...". At the bottom, there is a large blue button labeled "Analyse" with a small icon.

Challenges Faced

1. Data Quality and Limited Dataset
2. Text Preprocessing Complexity
3. Feature Extraction
4. Model Selection and Accuracy
5. Handling False Positives/Negatives
6. User Interface Design
7. Real-Time Prediction Performance
8. Error Handling
9. Model Integration with UI
10. Visualization of Results

Learnings & Key Takeaways

- Working with Real-Time Data
- Streamlit Application Development
- Data Parsing and Processing
- Error Handling and Debugging
- UI/UX Design Principles
- Code Organization and Maintainability
- Importance of Security Practices
- Performance Optimization
- Real-World Application Development

Future Enhancements

- Model Improvement & Hybridization
- Data & Scalability
- User Experience (UX) & Features
- Implement a User Feedback Mechanism
- Implement live data updates

References

- [Scikit-learn Documentation](#)
- [Streamlit Documentation](#)
- [Pandas Documentation](#)
- [NumPy Documentation](#)
- [Matplotlib Documentation](#)
- [Seaborn Documentation](#)